

PostgreSQL Disaster Recovery

Table of Contents

1. [Learning Objectives](#)
 2. [What Constitutes a Disaster](#)
 3. [DR Planning Framework](#)
 4. [RTO/RPO Planning](#)
 5. [DR Architecture Patterns](#)
 6. [Database Restore Runbook](#)
 7. [Scenario 1: Accidental Table Drop](#)
 8. [Scenario 2: Primary Server Hardware Failure](#)
 9. [Scenario 3: Complete Datacenter Loss](#)
 10. [Scenario 4: Database Corruption](#)
 11. [Testing DR Procedures](#)
 12. [Common Mistakes](#)
 13. [Best Practices](#)
 14. [Interview Questions](#)
 15. [Exercises and Solutions](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Classify disaster types and apply appropriate recovery strategies
 - Document RTO and RPO for different applications
 - Write a complete database restore runbook
 - Execute recovery for different disaster scenarios
 - Test DR procedures regularly
 - Calculate and justify DR infrastructure costs
-

What Constitutes a Disaster

DISASTER TAXONOMY:

	Data Lost?	Downtime?
Hardware Failure (disk, server)	Maybe	Yes
OS Corruption (unbootable)	No	Yes
Application Bug (bad UPDATE/DELETE)	Yes	Possible
Human Error (DROP TABLE, truncate)	Yes	Possible
Ransomware/Malware	Yes	Yes
Network Partition (split-brain)	Maybe	Maybe
Datacenter Outage (power, fire)	No*	Yes
Regional Disaster (flood, earthquake)	No*	Yes

* If DR standby in different region

THE KEY INSIGHT:

- Replication handles hardware failure (server crash)

- Backups handle data corruption (human error, bugs)
- Both together handle catastrophic datacenter loss

DR Planning Framework

DR Plan Components

1. RISK ASSESSMENT
 - What disasters can occur?
 - What is the probability?
 - What is the business impact?
2. RPO/RT0 TARGETS
 - Per application/database tier
 - Business-approved and SLA-aligned
3. RECOVERY PROCEDURES
 - Written, tested runbooks for each scenario
 - Contact list for escalation
4. INFRASTRUCTURE
 - DR environment (standby, backup storage)
 - Network routing (DNS, VIP, load balancer)
5. TESTING SCHEDULE
 - Quarterly tabletop exercises
 - Semi-annual actual recovery tests
6. COMMUNICATION PLAN
 - Who is notified and when
 - Status page updates
 - Customer communication templates

RTO/RPO Planning

APPLICATION TIERS:

TIER 1: Mission-Critical

Example: Payment processing, order management

RPO: 0 (no data loss)

RT0: < 5 minutes

Strategy: Patroni + sync replication + Patroni auto-failover

Infrastructure: 3+ PostgreSQL nodes, etcd cluster, HAProxy

TIER 2: Business-Critical

Example: Customer accounts, inventory

RPO: < 5 minutes

RT0: < 30 minutes

Strategy: Patroni + async replication + WAL archiving

Infrastructure: 2+ PostgreSQL nodes, Patroni, pgBackRest

TIER 3: Standard

Example: Reporting database, analytics

RPO: < 4 hours

RT0: < 2 hours

Strategy: Async streaming replica + daily pg_basebackup

Infrastructure: Primary + 1 replica, pgBackRest

TIER 4: Non-Critical

Example: Development/staging data copies

RPO: 24 hours

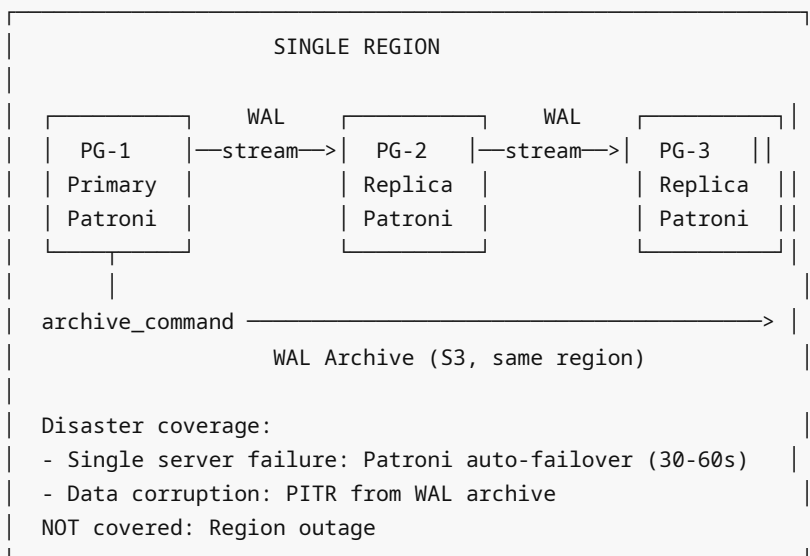
RT0: 8 hours

Strategy: Daily pg_dump

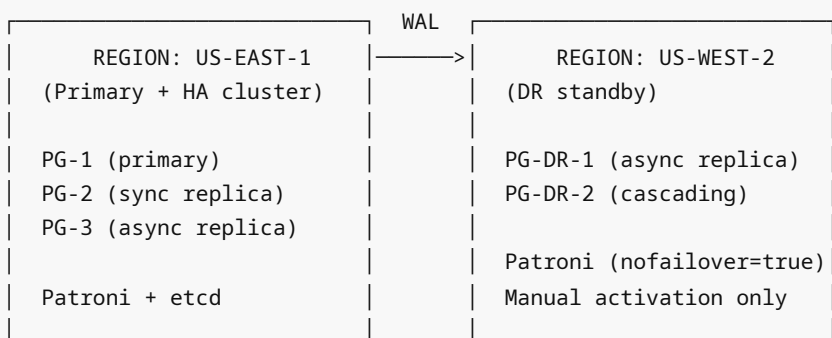
Infrastructure: Basic PostgreSQL, cron backup

DR Architecture Patterns

Pattern 1: Same-Region HA + Backups



Pattern 2: Multi-Region DR



[illegible]

- Status page: status.company.com → Investigating

=====

STEP 2: CONTAIN (5-15 minutes)

a) PREVENT FURTHER DAMAGE:

- If data is being deleted: STOP the application immediately
kubect1 scale deployment myapp --replicas=0
or: set maintenance mode
- If primary is failing: let Patroni promote (don't interfere)
- If single server hardware issue: isolate (don't fencing yet)

b) DOCUMENT:

- Time of incident
- Time first noticed
- Nature of incident
- Actions taken

=====

STEP 3: RECOVER (15-120 minutes)

→ Follow scenario-specific runbook below

=====

STEP 4: VERIFY (after recovery)

a) Verify data integrity:

```
psql -c "SELECT COUNT(*) FROM critical_table;"  
psql -c "SELECT MAX(created_at) FROM orders;"
```

b) Verify application connectivity:

Run smoke tests
Check error rate in APM

c) Verify backup resumed:

```
pgbackrest --stanza=mydb info  
psql -c "SELECT * FROM pg_stat_archiver;"
```

=====

STEP 5: COMMUNICATE

- a) Update status page: Resolved
- b) Notify stakeholders: All clear
- c) Document timeline: Incident report

=====

POST-INCIDENT (within 48 hours)

- a) Write incident report
- b) Root cause analysis
- c) Action items to prevent recurrence
- d) Update runbook if needed

e) Schedule follow-up test of affected scenario

=====

Scenario 1: Accidental Table Drop

Detected: Application errors: "table orders does not exist" **Root cause:** Developer ran `DROP TABLE orders;` in production

```
# IMMEDIATE: Stop all application writes
kubectl scale deployment orders-service --replicas=0
# or: maintenance mode

# ASSESS: Find when it happened
grep "DROP TABLE" /var/log/postgresql/postgresql-*.log
# Result: 2024-01-15 14:32:15 UTC DROP TABLE public.orders

# CHOOSE RECOVERY METHOD:
# Option A: PITR (preferred – minimal data loss)
# Option B: pg_dump restore (if PITR not available)

# == OPTION A: PITR ==
# On recovery server:
sudo systemctl stop postgresql@16-main
sudo -u postgres rm -rf /var/lib/postgresql/16/main/*

sudo -u postgres pgbackrest --stanza=mydb restore \
  --type=time \
  --target="2024-01-15 14:31:00+00" \
  --target-action=pause \
  --delta

sudo systemctl start postgresql@16-main

# Wait for recovery (watch logs)
tail -f /var/log/postgresql/postgresql-16-main.log
# Wait for: "pausing at the end of recovery"

# Verify orders table is intact
psql -h recovery-server -c "SELECT COUNT(*) FROM orders;"
# Result: 5,000,000 rows – confirmed!

# Export just the orders table
pg_dump -h recovery-server -U postgres -d mydb -t orders \
  -F c -f /tmp/orders_recovered.dump

# Restore orders table to production
psql -h production -c "TRUNCATE orders CASCADE;" 2>/dev/null || true
pg_restore -h production -U postgres -d mydb \
  --data-only --table=orders /tmp/orders_recovered.dump

# Restart application
```

```
kubectl scale deployment orders-service --replicas=3
```

```
# Verify
```

```
psql -h production -c "SELECT COUNT(*) FROM orders;"
```

Scenario 2: Primary Server Hardware Failure

Detected: Primary PostgreSQL server stops responding. Patroni promotes replica automatically.

```
# ASSESS: Is Patroni handling it automatically?
```

```
patronictl -c /etc/patroni/patroni.yml list
```

```
# Wait 30-60 seconds for automatic failover
```

```
# If Patroni works: new leader should appear
```

```
# Monitor: HAProxy routes to new primary automatically
```

```
curl -s http://haproxy:7000/stats
```

```
# Verify new primary
```

```
psql -h haproxy-vip -p 5000 -c "SELECT pg_is_in_recovery();" 
```

```
# Should return: f (false = primary)
```

```
# Re-sync old primary as new standby (when hardware is repaired):
```

```
# 1. Repair/replace hardware
```

```
# 2. Install PostgreSQL
```

```
# 3. Run pg_rewind (or pg_basebackup if diverged too far):
```

```
pg_rewind \
```

```
--target-pgdata=/var/lib/postgresql/16/main \
```

```
--source-server="host=new-primary port=5432 user=postgres dbname=postgres"
```

```
# Configure as standby
```

```
echo "primary_conninfo = 'host=new-primary ...'" >> postgresql.auto.conf
```

```
touch /var/lib/postgresql/16/main/standby.signal
```

```
# Start as standby
```

```
sudo systemctl start postgresql@16-main
```

```
# Verify it joined
```

```
patronictl list
```

Scenario 3: Complete Datacenter Loss

Detected: All servers in primary DC are unreachable.

```
# ASSESS: Is this truly a DC loss or a network partition?
```

```
ping primary-server.dc1.example.com
```

```
traceroute primary-server.dc1.example.com
```

```
# If no route: DC is down
```

```
# Check DR standby health
```

```

psql -h dr-standby.dc2.example.com -c "SELECT pg_is_in_recovery(),
pg_last_wal_replay_lsn();"

# Assess replication lag at time of failure (from monitoring)
# "Last known replication lag before DC went down: 45 seconds"
# This is potential data loss → inform stakeholders

# DECISION: Activate DR
# Requires: management approval (data loss acknowledgment)

# 1. Remove nofailover tag from DR Patroni
patronictl -c /etc/patroni/patroni_dr.yml edit-config production-db \
  --set tags.nofailover=false

# 2. Promote DR standby
patronictl -c /etc/patroni/patroni_dr.yml failover production-db \
  --master pg-dr-1 --force

# Verify promotion
psql -h pg-dr-1.dc2.example.com -c "SELECT pg_is_in_recovery();"
# Returns: f (primary now)

# 3. Update DNS / routing
aws route53 change-resource-record-sets \
  --hosted-zone-id Z123456 \
  --change-batch '{
    "Changes": [{
      "Action": "UPSERT",
      "ResourceRecordSet": {
        "Name": "db.example.com",
        "Type": "A",
        "TTL": 60,
        "ResourceRecords": [{"Value": "DR_SERVER_IP"}]
      }
    }]
  }'

# 4. Restart applications (new connection string)
# Applications using db.example.com will automatically reconnect

# 5. Document data loss
# "Approximately 45-120 seconds of transactions lost"
# "Orders 99999 - 100002 may be missing"

```

Scenario 4: Database Corruption

Detected: PostgreSQL crash recovery fails with corruption errors: LOG: database system identifier differs between pg_control and pg_filenode.map


```
# STOP immediately – do not attempt to start PostgreSQL
# Data files may be partially corrupt

# Check pg_control
pg_filedump /var/lib/postgresql/16/main/global/pg_control

# Try to get a list of corrupt files
sudo -u postgres postgres --single -j mydb << 'EOF'
SELECT * FROM pg_class LIMIT 1;
EOF

# RECOVERY: Use the most recent physical backup

# 1. Stop PostgreSQL
sudo systemctl stop postgresql@16-main

# 2. Preserve corrupt files for forensics (don't delete yet)
sudo mv /var/lib/postgresql/16/main /var/lib/postgresql/16/main.corrupted.$(date
+%Y%m%d)
sudo mkdir /var/lib/postgresql/16/main

# 3. Restore from pgBackRest (most recent backup + WAL)
sudo -u postgres pgbackrest --stanza=mydb restore --delta

# 4. Configure recovery to get up to the corruption event
# Get approximate time of corruption from logs
# Set recovery_target_time to just before corruption

sudo -u postgres cat >> /var/lib/postgresql/16/main/postgresql.auto.conf << 'EOF'
restore_command = 'pgbackrest --stanza=mydb archive-get %f %p'
recovery_target_time = '2024-01-15 15:30:00+00'
recovery_target_action = 'pause'
EOF

# 5. Start recovery
sudo systemctl start postgresql@16-main
tail -f /var/log/postgresql/*.log

# 6. Verify integrity
psql -h localhost -c "SELECT COUNT(*) FROM pg_class;"
psql -h localhost -c "SELECT schemaname, tablename FROM pg_tables WHERE schemaname =
'public';"

# 7. Run pg_dump to verify no further corruption
pg_dump -h localhost -U postgres -d mydb --schema-only | head -200

# 8. If data looks good: promote
psql -c "SELECT pg_wal_replay_resume();"

```

Testing DR Procedures

```
#!/bin/bash
# quarterly_dr_test.sh
# Run this quarterly to validate DR procedures

echo "=== QUARTERLY DR TEST ===" > /tmp/dr_test_report.txt
echo "Date: $(date)" >> /tmp/dr_test_report.txt

# Test 1: Backup freshness
echo "--- Test 1: Backup freshness ---" >> /tmp/dr_test_report.txt
BACKUP_AGE=$(pgbackrest --stanza=mydb info --output=json | \
    python3 -c "import json,sys; d=json.load(sys.stdin); print(d[0]['backup'][-1]
['timestamp']['stop'])")
echo "Last backup: $BACKUP_AGE" >> /tmp/dr_test_report.txt

# Test 2: WAL archiving
echo "--- Test 2: WAL archiving ---" >> /tmp/dr_test_report.txt
psql -c "SELECT last_archived_wal, last_archived_time, failed_count FROM
pg_stat_archiver;" \
    >> /tmp/dr_test_report.txt

# Test 3: Restore to test environment
echo "--- Test 3: Restore test ---" >> /tmp/dr_test_report.txt
TEST_PGDATA=/var/lib/postgresql/dr_test_$(date +%Y%m%d)
mkdir -p "$TEST_PGDATA"
TIME_BEFORE=$(date +%s)

pgbackrest --stanza=mydb restore \
    --pg1-path="$TEST_PGDATA" \
    --type=time \
    --target="$(date -d '1 hour ago' '+%Y-%m-%d %H:%M:%S %Z')" \
    --target-action=promote

TIME_AFTER=$(date +%s)
RESTORE_TIME=$((TIME_AFTER - TIME_BEFORE))
echo "Restore completed in ${RESTORE_TIME} seconds" >> /tmp/dr_test_report.txt

# Start test instance
pg_ctl start -D "$TEST_PGDATA" -o "-p 5435" -l /tmp/dr_test.log

# Test 4: Verify data
echo "--- Test 4: Data verification ---" >> /tmp/dr_test_report.txt
psql -h localhost -p 5435 -U postgres -d mydb -c \
    "SELECT schemaname, tablename,
pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS size
FROM pg_tables WHERE schemaname = 'public'
ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC
LIMIT 10;" >> /tmp/dr_test_report.txt
```

```
# Cleanup
pg_ctl stop -D "$TEST_PGDATA" -m fast
rm -rf "$TEST_PGDATA"

echo "DR test completed. Report: /tmp/dr_test_report.txt"
cat /tmp/dr_test_report.txt | mail -s "Quarterly DR Test Report" dba@company.com
```

Common Mistakes

1. **No DR plan until disaster strikes** — improvising under pressure causes more errors
 2. **Untested backups** — backup exists but restore fails
 3. **Restoring to production instead of test server** — double disaster
 4. **No data loss acknowledgment before DR activation** — stakeholders surprised by RPO
 5. **Forgetting sequence values** after PITR — new inserts get duplicate key errors
 6. **Not updating DNS/connection strings** after DR activation
 7. **Panicking and not following the runbook** — rushing causes mistakes
 8. **No post-mortem** — same disaster happens again
 9. **DR environment not tested regularly** — servers offline, software outdated
 10. **Recovery time estimates not validated** — RTO SLA impossible to meet
-

Best Practices

1. **Maintain a written runbook** — updated and version-controlled
 2. **Test DR quarterly** — actual restore, not just "backup is running"
 3. **Recover to a SEPARATE server first** — never overwrite production without verification
 4. **Communicate RTO/RPO to stakeholders** — manage expectations
 5. **Automate health checks** — alert before disasters compound
 6. **Create named restore points** before risky operations
 7. **Store runbook outside the database** — can't read runbook if DB is down
 8. **Practice the human elements** — who to call, who approves DR activation
 9. **Document actual RTO** from DR tests — know if you can meet the SLA
 10. **Review and update DR plan** after every incident
-

Interview Questions

Q1: What is the first thing you do when the production database goes down?

A: Follow the runbook: (1) Check monitoring — is it truly down or just one alert? (2) If Patroni is present: wait 30-60 seconds for automatic failover. (3) Stop applications from generating more errors/data loss while assessing. (4) Notify stakeholders immediately. (5) Classify the disaster type (hardware failure, data corruption, etc.) to apply the right recovery procedure. NEVER make hasty changes without understanding the situation.

Q2: You drop a table with 10 million rows in production. What do you do?

A: (1) Immediately stop application writes to prevent new data from filling the RPO gap. (2) Check WAL archiving is current (`pg_stat_archiver`). (3) Find the DROP TABLE timestamp from PostgreSQL logs. (4) On a separate recovery server, restore the latest pgBackRest base backup. (5) Configure `recovery_target_time` to 1 minute before the DROP. (6) Start recovery with `target_action=pause` . (7) Verify table exists and count rows. (8) Export the table to SQL. (9) Restore to production. (10) Write incident report.

Q3: What's the difference between RPO and RTO and how do they drive infrastructure decisions?

A: RPO (Recovery Point Objective) is maximum acceptable data loss — how far back in time can you restore without unacceptable business impact. RTO (Recovery Time Objective) is maximum acceptable downtime — how long can the service be unavailable. RPO drives backup frequency and WAL archiving (RPO=0 → sync replication; RPO=5min → WAL archiving with `archive_timeout=300`). RTO drives recovery speed: physical backups + parallel restore for low RTO; `pg_dump` for higher RTO.

Q4: How do you activate a DR standby in another region?

A: (1) Verify the primary DC is truly unreachable (not network partition). (2) Assess replication lag on DR standby — inform stakeholders of potential data loss. (3) Get approval for data loss acceptance. (4) If using Patroni: change `nofailover` tag to false, trigger failover. (5) Update DNS/VIP to point to DR region. (6) Restart applications or update connection strings. (7) Verify application connects to DR.

Q5: After a major recovery, what steps should you follow?

A: (1) Verify data integrity (counts, recent records). (2) Run smoke tests / application health checks. (3) Verify backup resumes (archiving, `pgBackRest` backup). (4) Verify replication resumes (if applicable). (5) Update status page. (6) Notify all stakeholders. (7) Monitor for 2-4 hours for stability. (8) Write incident timeline. (9) Schedule post-mortem within 48 hours.

Exercises and Solutions

Exercise 1: Create DR Runbook for Your Database

Template to fill out:

```
DATABASE: [name]
VERSION: PostgreSQL [X.Y]
SIZE: [N] GB
TIER: [1/2/3/4]
RPO: [N] minutes
RTO: [N] minutes

BACKUP CONFIGURATION:
  Tool: [pgBackRest/pg_dump]
  Schedule: [cron expression]
  Storage: [path/S3 bucket]
  Last verified: [date]

REPLICATION:
  Standbys: [IP addresses]
  Patroni: [yes/no]
  Sync standby: [yes/no]

RECOVERY CONTACTS:
  Primary: [name, phone]
  Secondary: [name, phone]

RECOVERY COMMANDS:
  [Fill in with actual commands for your environment]
```

Cross-References

- [04_pitr.md](#) — PITR for data recovery
- [06_pgbackrest.md](#) — pgBackRest restore commands
- [08_backup_monitoring.md](#) — Detecting backup issues proactively
- [../13_Replication_HA/05_failover_procedures.md](#) — Server failure failover
- [../13_Replication_HA/09_ha_architecture_patterns.md](#) — DR architecture